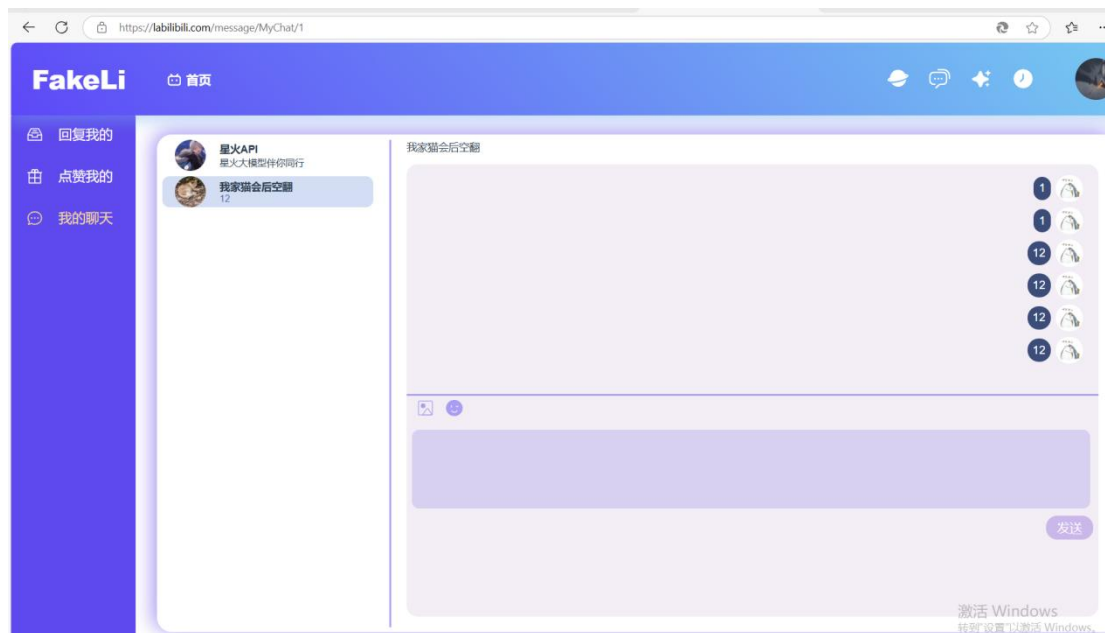
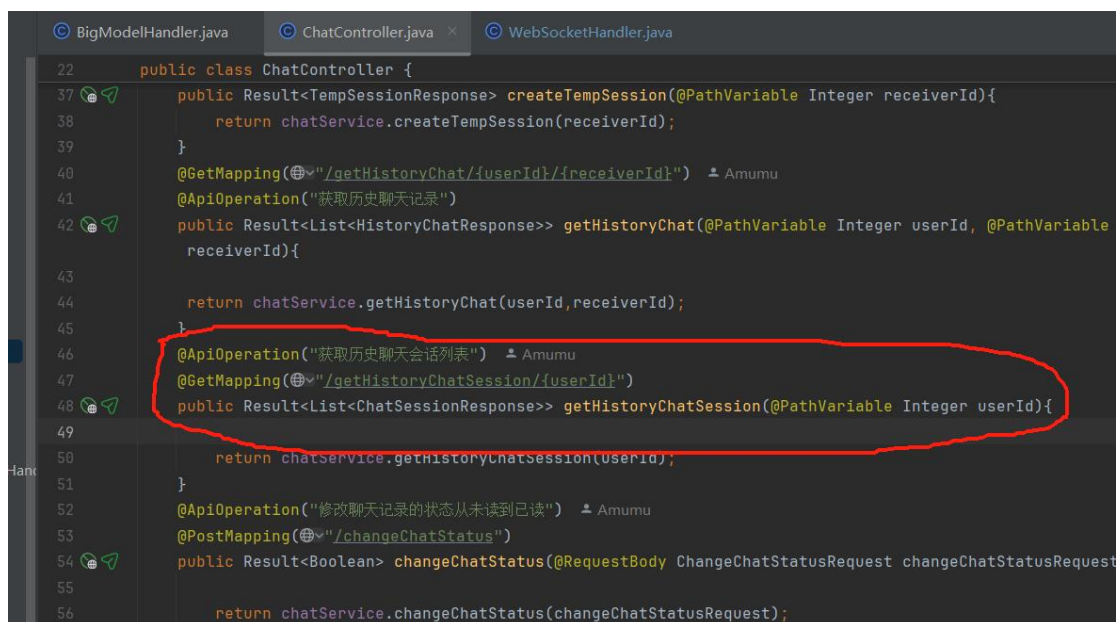




私聊

进入页面后会发送请求



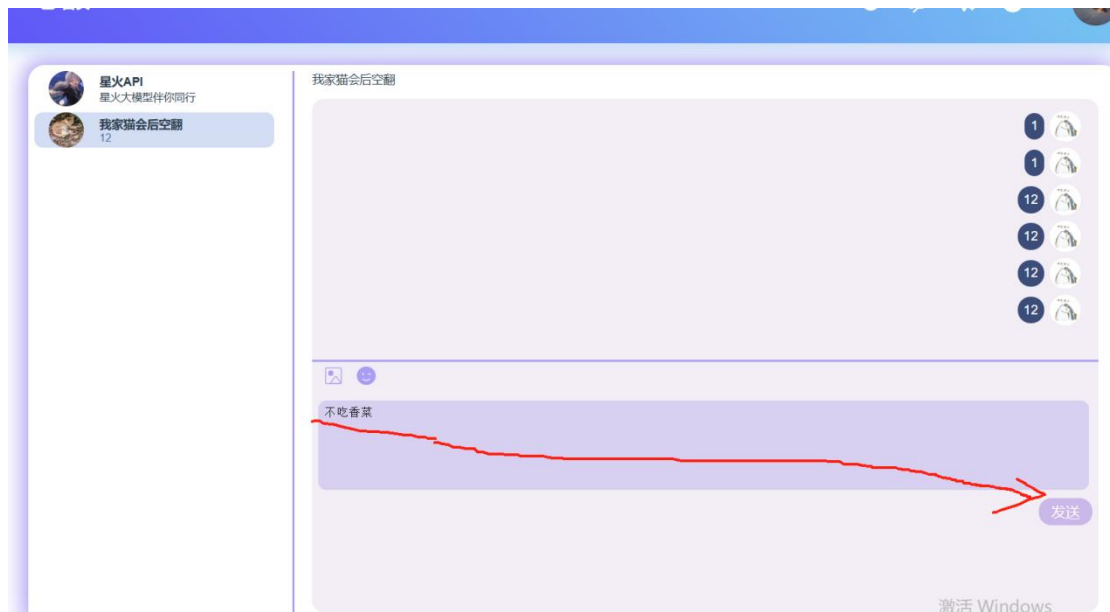
获取历史聊天会话列表，并默认选中第一个，同时发送申请建立 websocket 连接
连接建立后前端发送一条 websocket 消息，将前端的 userid 传给后端，触发后端接收消息方法，将该客户端的 userid 和 websocketsession 对象进行映射绑定

```

public void handleTextMessage(WebSocketSession session, TextMessage message) {
    //将消息json化
    JsonObject json = JsonParser.parseString(message.getPayload()).getAsJsonObject();
    String type = json.get(MESSAGE_TYPE).getAsString();
    //接收消息后根据消息类型来处理
    switch (type) {
        //若是大模型则是将消息中的提问部分发送给大模型然后获取到大模型响应后返回给客户端
        case MESSAGE_TYPE_BIGMODEL:
            String question = json.get(MESSAGE_TYPE_BIGMODEL_QUESTION).getAsString();
            String id = json.get(USER_IDENTITY).getAsString();
            if (BIGMODEL_MAP.get(id) != null) {
                BIGMODEL_MAP.get(id).send(question, id);
            } else {
                BigModelHandler bigModelHandler = new BigModelHandler(applicationContext);
                BIGMODEL_MAP.put(id, bigModelHandler);
                bigModelHandler.send(question, id);
            }
            break;
        //若是初始化则是将userId和sessionId的映射存到map里
        case MESSAGE_TYPE_INIT:
            log.info("初始化");
            String userId = json.get(USER_IDENTITY).getAsString();
            WEB_SOCKET_SESSION_CONCURRENT_MAP.put(userId, session);
            break;
    }
}

```

之后发送消息时会将接收消息方的 userid 和消息内容传递给后端



调用接收消息方法，根据接收方的 `userid` 找到接收方的 `websocketsession` 对象
用该对象给需要接收消息的客户端发送 `websocket` 消息并持久化消息到数据库
前端接收到该 `websocket` 消息后立即渲染到页面上
若找不到该 `userid` 对应的 `websocketsession` 对象则说明该接收消息方不在线
后端不做转发到另一个客户端的操作，只将私聊消息存到数据库中，下图对应的回复我的消息中会有未读消息数

